

Giải Thích Chi Tiết Cách Hệ Thống Xử Lý Video Và Nhúng Tin

H.264/CAVLC: kênh dấu trailing-one, lịch nhúng có khóa và proof Groth16

Tài liệu kỹ thuật cho dự án

Cập nhật ngày 03/10/2026

Cập nhật 03/10/2026. Tài liệu được viết lại theo hệ thống hiện tại (nguồn đối chiếu: doc/he_thong_hoat_dong.md, README.md, demo/README.md và mã nguồn). Những thay đổi chính so với bản tháng 9/2026:

- Chỉ còn **một lỗi media native C++** (native/). Pipeline Python cũ (parser/patcher Python, bộ lọc vị trí nhiều tầng, file phụ vị trí, benchmark SEC1–SEC10) đã bị gỡ ngày 03/10/2026; các khái niệm “safety filter”, “patchability pruning”, “capacity funnel” trong bản cũ không còn tồn tại.
- Vị trí nhúng do **lịch HMAC có khóa** quyết định; trích xuất là **mù** (không cần video gốc, không cần file phụ).
- Giao thức kênh **v2**: khóa con HKDF-SHA256, khung [0x02][len 2B][payload][tag 16B], làm trắng bit bằng keystream HMAC. Một giao thức theo đoạn IDR dùng chung cho file, job HTTP và luồng live.
- Mạch PayloadVerify: 63 321 constraint, ràng buộc 0/1 cho payload_hash, commitment, secret; 513 public input.
- Dịch vụ HTTP: job verify bắt buộc Groth16; demo duy nhất demo/terminal_demo.py; run_all.py 126/126 qua 12 phase.

1 Câu Trả Lời Ngắn Gọn Trước

Nếu nói thật ngắn, hệ thống làm việc như sau:

*Nó không sửa pixel của video. Nó đặt lại giá trị của một số bit **trailing_ones_sign_flag** — bit dấu của hệ số trailing-one trong các khối residual CAVLC của slice IDR — tại những vị trí do một khóa bí mật chọn.*

Một file H.264 được lỗi native nhìn như một cấu trúc nhiều tầng:

file .h264 → NAL units → IDR slices → macroblocks → residual blocks → bit dấu trailing-one

Mỗi khối residual có hệ số trailing-one cho đúng **một ứng viên**: bit dấu của trailing-one đầu tiên. Lật bit này không đổi độ dài mã, không đổi ngữ cảnh CAVLC, nên bitstream luôn hợp lệ mà không cần kiểm tra “vá được hay không”. Việc còn lại là chọn ứng viên nào mang bit nào: hệ thống dùng HMAC theo khóa để xếp hạng ứng viên, rồi ghi một khung đã xác thực (HMAC) và đã làm trắng vào các vị trí đứng đầu. Bên nhận có cùng khóa tự dựng lại đúng lịch đó từ video stego.

Ba lớp có trách nhiệm tách biệt:

Lớp	Thực hiện ở	Bảo đảm
Media	<code>native/src/cavlc_stream.cpp</code>	Bitstream sau nhúng vẫn hợp lệ, độ dài mã không đổi, decoder chuẩn giải được.
Kênh xác thực	<code>native/src/cavlc_stream.cpp</code> , <code>src/native_blind_contract.py</code>	Chỉ người có khóa tìm được vị trí; sai khóa hoặc sửa bit thì HMAC từ chối.
Proof	<code>src/zk_proof.py</code> , <code>circuits/payload_verify.circom</code>	Quan hệ commitment = $\text{SHA256}(\text{SHA256}(\text{message}) \parallel \text{secret})$ được chứng minh bằng Groth16.

2 Codec, H.264 Và Vì Sao Hệ Thống Không Sửa Pixel

2.1 Codec là gì?

Codec là bộ mã hóa/giải mã video. Tên này đến từ:

$$\text{coder} + \text{decoder} = \text{codec}$$

Video thô rất lớn. Codec như H.264 nén video thành bitstream nhỏ hơn để lưu, truyền, hoặc phát lại. Khi phát video, decoder đọc bitstream đó và dựng lại các frame.

2.2 Sửa pixel và sửa bitstream khác nhau thế nào?

Một hệ đơn giản có thể làm như sau:

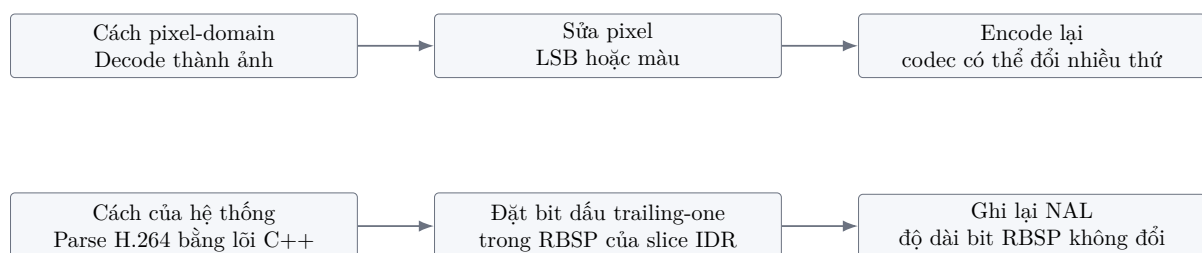
decode video -> sửa pixel -> encode lại video

Đó là cách làm ở **pixel domain**. Nó dễ hiểu nhưng có một vấn đề lớn: khi encode lại, codec có thể thay đổi rất nhiều thứ. Bit nhúng có thể mất, artifact có thể xuất hiện, và khó kiểm soát chính xác bitstream cuối cùng.

Hệ thống này đi theo hướng khác:

parse H.264 (native) -> đặt bit dấu trong RBSP -> ghi lại NAL

Đây là cách làm ở **compressed domain**. Không có bước encode lại: chỉ các bit dấu được chọn thay giá trị, mọi bit khác của file giữ nguyên (ngoại trừ emulation-prevention byte được tính lại, xem mục 4.3).



Hình 1: Khác biệt giữa sửa pixel và sửa dữ liệu nén trong H.264.

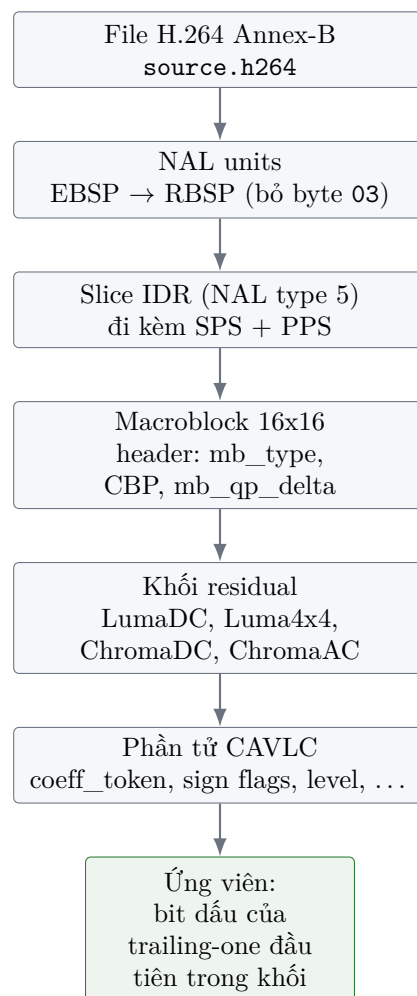
2.3 Video được chuẩn hóa về Baseline/CAVLC

Lõi native chỉ hiểu một tập con H.264: Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id. Vì vậy video được encode bằng FFmpeg/libx264 với tham số cố định (lệnh của demo, hàm `video_prepare` trong `demo/terminal_demo.py`):

Tham số	Lý do
<code>-profile:v baseline, cabac=0</code>	Parser native chỉ hỗ trợ CAVLC; CABAC không có kênh dấu độc lập như vậy.
<code>-g 1 / keyint=1</code>	Mọi frame là IDR. Chỉ IDR mang dữ liệu; GOP>1 vẫn chạy nhưng P-frame không có ứng viên.
<code>slices=1, threads=1</code>	Parser native chỉ nhận một slice mỗi frame.
<code>-bf 0</code>	Không B-frame.
<code>-qp 22</code>	Lượng tử cố định; x264 hạ QP của I-frame theo <code>ip_ratio=1.40</code> , nên slice có QP 19.

3 File H.264 Được Hệ Thống Bóc Ra Như Thế Nào?

Phần này là lỗi để nhìn rõ hệ thống. Khi nhận một file `source.h264`, lõi native không nhìn nó như một chuỗi byte vô nghĩa. Nó bóc file theo các tầng sau.



Hình 2: Tầng xử lý từ file H.264 đến ứng viên nhúng.

Hàm native `serialize_cavlc_sign_candidate` gán cho mỗi ứng viên một định danh dạng chuỗi ASCII:

```
nal_index:macroblock_address:category:block_index:rbp_bit_offset
```

Ví dụ trong `doc/he_thong_hoat_dong.md`:

Nghĩa là:

- NAL thứ 24 (trong đầu vào đang phân tích),
- macroblock địa chỉ 326 trong slice,
- category 1 = Luma4x4 (0 = LumaDC, 2 = ChromaDC, 3 = ChromaAC),
- khối số 6 trong macroblock,
- bit dấu nằm ở offset bit 67801 của RBSP.

Định danh này chính là đầu vào của hàm score HMAC (mục 8). Trong giao thức theo đoạn, định danh được tính theo đầu vào của đoạn (SPS+PPS+IDR), không theo vị trí trong toàn file.

4 Ví Dụ Đầy Đủ: Một Video H.264 Được Parse Ra Gì?

Các giá trị dưới đây là **giá trị thật** lấy từ một lần chạy `demo/terminal_demo.py` trên `data/raw/foreman_cif.y4m` (CIF 352x288, 8 frame, QP 22, GOP 1), như được ghi trong `doc/he_thong_hoat_dong.md`. Có thể tự quan sát chúng bằng `zkstego_inspect` hoặc `demo`.

4.1 File H.264 ban đầu trông như thế nào?

Một file `.h264` thô là chuỗi NAL unit, mỗi NAL bắt đầu bằng start code `00 00 01` hoặc `00 00 00 01`. Byte đầu tiên sau start code là NAL header: 1 bit `forbidden`, 2 bit `nal_ref_idc`, 5 bit `nal_unit_type`.

4.2 Kết quả parse NAL thật

Bảng NAL đầu của `source.h264` trong `demo`:

NAL	Offset	Start code	Header	Type	EBSP	RBSP	EPB
0	0	00000001	01100111	7 SPS	21	20	1
1	26	00000001	01101000	8 PPS	4	4	0
2	35	000001	00000110	6 SEI	568	568	0
3	607	000001	01100101	5 IDR	8797	8797	0

SPS/PPS cần để hiểu stream; SEI chứa chuỗi cấu hình của x264; chỉ NAL type 5 (slice IDR) mang dữ liệu nhúng.

4.3 EBSP, RBSP và emulation-prevention byte

Bên trong NAL, chuỗi `00 00 00`, `00 00 01`, `00 00 02`, `00 00 03` không được xuất hiện. Encoder chèn byte `03` sau mỗi cặp `00 00` đứng trước byte ≤ 3 . Payload có byte chèn gọi là **EBSP**; bỏ các byte đó thu được **RBSP**. Ví dụ thật trong SPS: `04 40 00 00 03 00 40 00` → RBSP `04 40 00 00 00 40 00`.

Mọi offset bit của parser native tính trên RBSP. Lật một bit RBSP có thể tạo hoặc xóa một mẫu `00 00 0x`, nên khi ghi lại NAL, EPB được tính lại (`rbsp_to_esp`). Độ dài **bit RBSP** không đổi, nhưng độ dài **byte EBSP** có thể lệch vài byte.

4.4 SPS, PPS và slice header

Trích từ `trace_headers` của FFmpeg trên clip demo (vị trí bit tính từ đầu NAL, kể cả 8 bit NAL header):

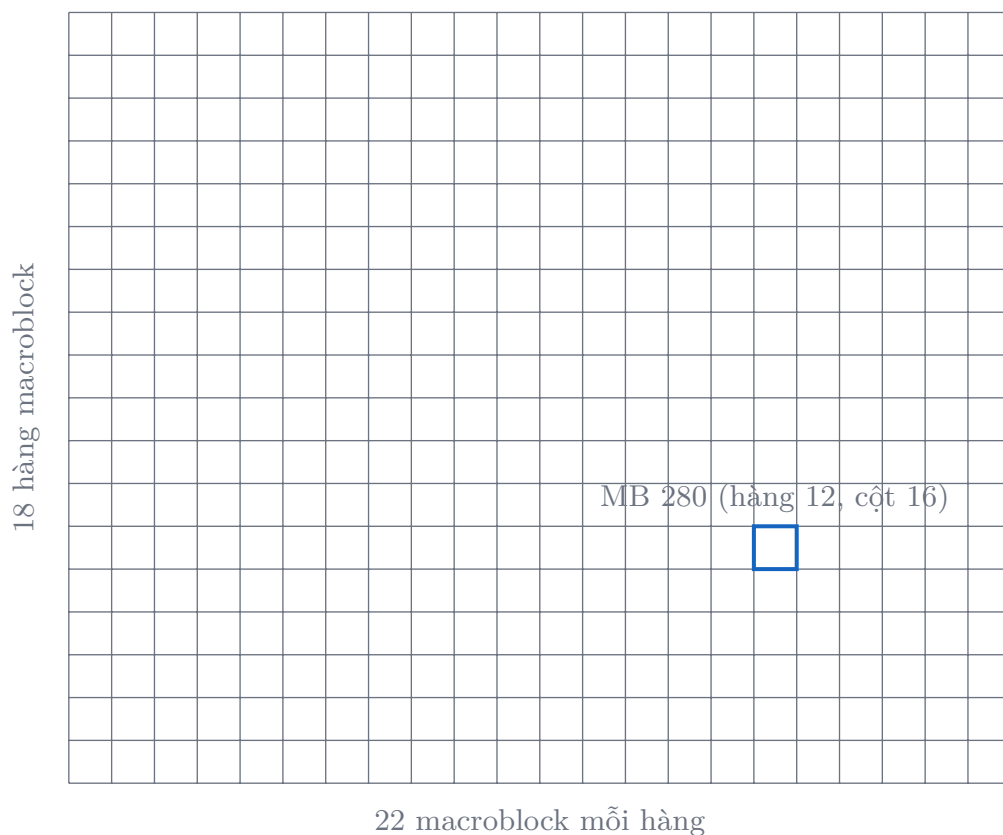
NAL	Trường	Giá trị
SPS	<code>profile_idc</code>	66 (Baseline)
SPS	<code>pic_width_in_mbs_minus1</code>	21 $\rightarrow$ 22 MB = 352 px
SPS	<code>pic_height_in_map_units_minus1</code>	17 $\rightarrow$ 18 MB = 288 px
PPS	<code>entropy_coding_mode_flag</code>	0 (CAVLC)
PPS	<code>pic_init_qp_minus26</code>	-4 $\rightarrow$ <code>pic_init_qp</code> 22
IDR	<code>slice_type</code>	7 (I)
IDR	<code>slice_qp_delta</code>	-3 $\rightarrow$ QP slice 19
IDR	<code>disable_deblocking_filter_idc</code>	0 (deblocking bật)

Slice header kết thúc ở bit 32 của NAL, tức bit 24 của RBSP; parser native báo `data_bit_offset = 24`. Demo kiểm tra hai con số này khớp nhau.

4.5 Từ IDR slice đến macroblock

Sau slice header là chuỗi macroblock theo thứ tự quét raster. Với CIF 352x288, mỗi frame có:

22 macroblock theo chiều ngang $\times$ 18 macroblock theo chiều dọc = 396 macroblock/frame.



Hình 3: Một frame CIF 352x288 được chia thành lưới 22x18 macroblock. Ô được tô là MB 280 ($280 = 12 \cdot 22 + 16$), macroblock dùng trong ví dụ bên dưới.

Header của một MB trong I-slice gồm `mb_type` (0 = I4x4, 1..24 = I16x16), 16 cờ/mode dự đoán Intra4x4 (nếu I4x4), `intra_chroma_pred_mode`, `coded_block_pattern` và `mb_qp_delta`. Ví dụ thật, MB 280 (đọc lại từng bit bởi `demo/h264_explain.py`):

Bit	Độ dài	Chuỗi bit	Phần tử = giá trị
48847	1	1	<code>mb_type = 0</code> (I4x4)
48848	1	1	<code>prev_intra4x4_pred_mode_flag[0] = 1</code>
48850	1	0	<code>prev_intra4x4_pred_mode_flag[2] = 0</code>
48851	3	001	<code>rem_intra4x4_pred_mode[2] = 1</code>
...			
48888	1	1	<code>intra_chroma_pred_mode = 0</code>
48889	1	1	<code>coded_block_pattern = 47</code>
48890	1	1	<code>mb_qp_delta = 0</code>

Với CQP và `aq=0`, mọi `mb_qp_delta` = 0 nên QP mọi MB = 19; demo đối chiếu với bản đồ QP do decoder FFMpeg in ra (396/396 khớp).

4.6 Từ macroblock đến các khối 4x4

Một MB I4x4 có 16 khối luma 4x4, đánh số `luma4x4BlkIdx` theo zig-zag của khối 8x8 (không theo thứ tự raster):

0	1	4	5
2	3	6	7
8	9	12	13
10	11	14	15

Hình 4: Chỉ số `luma4x4BlkIdx` của 16 khối luma 4x4 trong một macroblock.

Ngoài 16 khối Luma4x4, residual của MB còn có LumaDC (chỉ với I16x16, 16 hệ số DC qua Hadamard; khi đó mỗi khối luma chỉ có 15 hệ số AC), ChromaDC (2x2 cho mỗi plane U/V) và ChromaAC (15 hệ số mỗi khối, 8 khối). Cả bốn loại đều có thể sinh ứng viên.

4.7 Một khối 4x4 sau khi parse trông như thế nào?

Ví dụ thật: frame #6, MB 280, khối `luma4x4BlkIdx` 2. Hệ số theo thứ tự zig-zag trước nhúng là [2, -1, 0, ...]; -1 là trailing-one. Ma trận hệ số:

2	-1	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Với mỗi khối, `zkstego_inspect --macroblock NAL MB` xuất các trường (tên khóa JSON thật của công cụ):

Trường	Ý nghĩa
category, index	Loại khối và chỉ số khối trong MB.
n_c	Ngữ cảnh nC dùng để chọn bảng <code>coeff_token</code> .
total_coeff, trailing_ones	Số hệ số khác 0 và số trailing-one.
coeff_token_bit, end_bit	Vị trí bit bắt đầu <code>coeff_token</code> và bit kết thúc khối trong RBSP.
sign_offsets	Offset bit RBSP của các <code>trailing_ones_sign_flag</code> .
coefficients_scan	Hệ số theo thứ tự quét.

Ở bước này chưa có gì được nhúng. Parser chỉ liệt kê: khối nào có trailing-one và bit dấu đầu tiên nằm ở offset nào.

5 Residual Coefficient Là Gì?

H.264 thường không lưu trực tiếp từng pixel. Nó dự đoán nội dung của một khối từ các mẫu **đã tái tạo** bên trái/trên (dự đoán intra), rồi lưu phần sai khác giữa dự đoán và dữ liệu thật. Phần sai khác này gọi là **residual**. Sau biến đổi nguyên 4x4 (xấp xỉ DCT) và lượng tử hóa theo QP, residual trở thành các số nguyên gọi là **residual coefficients**, được quét zig-zag rồi mã hóa entropy bằng CAVLC.

Phía decoder làm ngược lại: giải lượng tử, biến đổi ngược, cộng với dự đoán, rồi áp deblocking cho cả frame. Hệ số trong ma trận **không phải pixel**: đổi một hệ số làm đổi cả khối residual 4x4 sau biến đổi ngược.

Hệ thống không sửa giá trị tuyệt đối của bất kỳ hệ số nào. Nó chỉ đổi **dấu** của một hệ số ± 1 nằm ở cuối dãy hệ số khác 0, vì dấu này được mã hóa bằng một bit riêng: `trailing_ones_sign_flag`.

6 CAVLC Và Trailing Ones

6.1 CAVLC là gì?

CAVLC là viết tắt của:

Context-Adaptive Variable Length Coding

Đây là cách H.264 Baseline mã hóa residual coefficients. Mỗi khối được mã thành 5 loại phần tử, đúng thứ tự (mục 9.2 của chuẩn):

Phần tử	Nội dung
<code>coeff_token</code>	Cặp (TotalCoeff, TrailingOnes); bảng VLC chọn theo nC.
<code>trailing_ones_sign_flag</code>	Mỗi trailing-one 1 bit dấu: 0 $\rightarrow$ +1, 1 $\rightarrow$ -1. Đây là kênh nhúng.
<code>level</code>	Các hệ số còn lại, giải theo thứ tự ngược scan (<code>level_prefix</code> + <code>level_suffix</code>).
<code>total_zeros</code>	Tổng số 0 đứng trước hệ số khác 0 cuối cùng.
<code>run_before</code>	Số 0 ngay trước mỗi hệ số.

nC là trung bình TotalCoeff của khối bên trái (nA) và bên trên (nB), làm tròn lên; ChromaDC dùng $nC = -1$. Vì nC của khối sau phụ thuộc TotalCoeff của khối trước, sửa *giá trị* hệ số bất kỳ có thể đổi bảng VLC và làm lệch toàn bộ phần còn lại của slice. Đó là lý do hệ thống chỉ chạm vào bit dấu.

6.2 Trailing ones là gì?

Trong CAVLC, tối đa ba hệ số cuối có trị tuyệt đối bằng 1 được mã hóa đặc biệt: chúng là **trailing ones** (T1). Độ lớn của chúng nằm sẵn trong `coeff_token`; chỉ dấu được ghi bằng một bit cố định.

Ví dụ thật từ demo (khối có $nC = 4$, bằng $4 \leq nC < 8$):

Bit	Độ dài	Chuỗi bit	Phần tử = giá trị
34636	4	1110	<code>coeff_token</code> = (1, 1): TotalCoeff 1, TrailingOnes 1
34640	1	1	<code>trailing_ones_sign_flag</code> [0] = 1 $\rightarrow$ hệ số -1 bit nhúng
34641	3	011	<code>total_zeros</code> = 1

Sau khi nhúng: bit 34640 = 0 $\rightarrow$ hệ số $+1$; mọi bit khác giữ nguyên. `demo/h264_explain.py` đọc lại đọc lập từng phần tử và kiểm tra việc đọc dừng đúng ở bit kết thúc do parser native báo.

6.3 Vì sao lật dấu trailing-one luôn an toàn về cú pháp

- **Không đổi độ dài mã:** chỉ một bit cố định thay giá trị; mọi phần tử sau giữ nguyên vị trí.
- **Không đổi ngữ cảnh:** TotalCoeff và TrailingOnes không đổi nên nC của các khối sau, bảng VLC được chọn và toàn bộ cú pháp phía sau giữ nguyên.
- **Thay đổi pixel nhỏ:** một hệ số đổi từ ± 1 sang ∓ 1 ($\Delta = \pm 2$ ở mức lượng tử).

Vì tính chất này đúng *theo cấu tạo*, lõi native không cần bước “thử sửa rồi mã hóa lại khối để xem có vá được không”. Mỗi ứng viên đều dùng được.

7 Ví Dụ Cụ Thể: Một Bit Đi Vào Một Vị Trí

Bit thực sự được ghi vào video không phải bit payload thô. Chuỗi biến đổi là:

$$\begin{aligned} \text{payload} &\rightarrow \text{khung } v2 \rightarrow f_i \text{ (bit khung, MSB trước)} \\ &\rightarrow e_i = f_i \oplus k_i \rightarrow \text{trailing_ones_sign_flag} := e_i \end{aligned}$$

trong đó k_i là bit thứ i của keystream làm trắng (mục 10.2). Bit nhúng 0 $\rightarrow$ dấu $+$, bit nhúng 1 $\rightarrow$ dấu $-$, đúng quy ước của `trailing_ones_sign_flag`. Lõi native **đặt** bit (không “lật” vô điều kiện): nếu dấu gốc đã trùng với e_i , bit đó không đổi.

Bảng minh họa dạng ký hiệu (ứng viên c_j là ứng viên đứng hạng j trong lịch của đoạn):

Bit khung i	f_i	k_i	e_i	Ứng viên	Dấu sau nhúng
0	0	1	1	c_0	-1
1	0	0	0	c_1	$+1$
2	1	1	0	c_2	$+1$
3	1	0	1	c_3	-1

Demo (bước “Bit nhúng nằm ở đâu”) in đúng chuỗi này cho một bit thật: định danh ứng viên, score HMAC, thứ hạng, chỉ số bit trong khung, trường mang bit (version/độ dài/message/A.x/B.x/C.x/tag), byte RBSP, byte EBSP (đếm EPB), offset trong file và hex trước/sau; rồi kiểm tra bit trong stego bằng bit khung XOR bit keystream.

8 Lịch Nhúng Có Khóa Theo Đoạn IDR

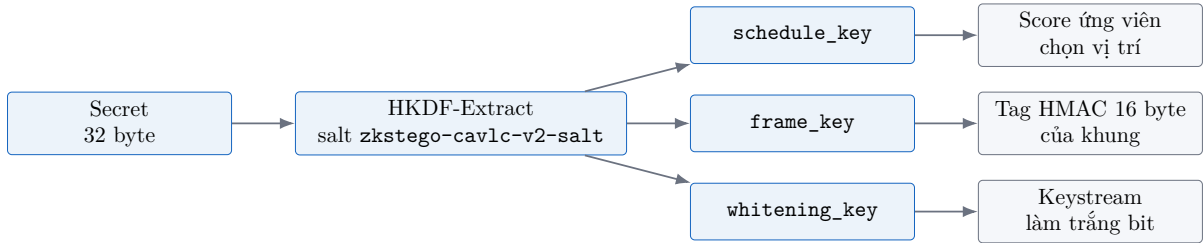
Vị trí nhận từng bit không cố định mà do khóa quyết định, để chỉ người có khóa đọc được. Đây là phần thay thế toàn bộ “pipeline chọn vị trí” của bản tài liệu cũ.

8.1 Khóa con HKDF (giao thức v2)

Mỗi vai trò dùng một khóa con riêng, sinh bằng HKDF-SHA256 (RFC 5869) từ secret 32 byte:

```
PRK = HMAC-SHA256(key = "zkstego-cavlc-v2-salt", msg = secret)
schedule_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/schedule", 32)
frame_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/frame-tag", 32)
whitening_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/whitening", 32)
```

Lộ một khóa con (ví dụ khóa lịch) không làm lộ secret hay các khóa con khác.



Hình 5: Ba khóa con của giao thức v2 và vai trò của chúng.

8.2 Score và thứ tự ứng viên

$\text{score}(c) = \text{HMAC-SHA256}(\text{schedule_key}, \text{định danh ASCII của } c)$

Ứng viên được sắp xếp tăng dần theo (score, định danh); lấy N ứng viên đầu. Bit thứ i của khung (sau khi làm trắng) gán cho ứng viên thứ i .

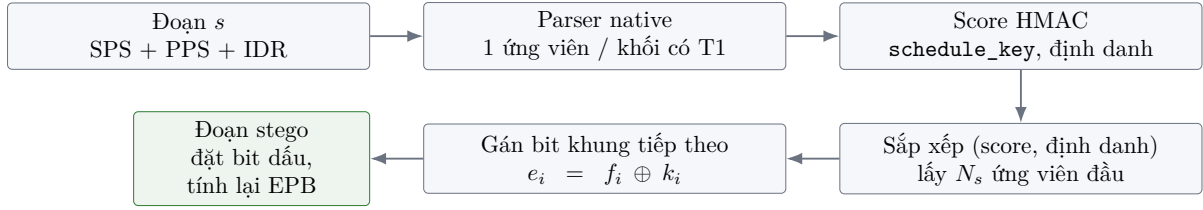
- Native: `score_keyed_cavlc_sign_candidate` và `select_keyed_cavlc_sign_candidates`.
- Python tham chiếu cùng ABI (có test vector chéo ngôn ngữ): `src/native_blind_contract.py`.

8.3 Một giao thức theo đoạn

Chỉ có một giao thức, **theo đoạn** (segment), dùng chung cho file, job HTTP và luồng live:

- mỗi đoạn = SPS + PPS + 1 IDR;
- đoạn s mang tối đa $N_s = \min(\text{max_bits_per_IDR}, \text{số ứng viên của } s)$ bit tiếp theo của khung, chọn theo lịch HMAC của chính đoạn đó;
- khung trải qua nhiều IDR liên tiếp; chỉ số bit khung đếm liên tục qua các đoạn;
- ở đoạn cuối, các vị trí trong lịch vượt quá cuối khung giữ nguyên dấu gốc.

Bên nhận phải dùng cùng `max_bits_per_IDR`, nếu không lệch lệch và trích sai. Dịch vụ dùng `ZK_STEGO_MAX_BITS_PER_IDR` (mặc định 64); demo dùng `--max-bits-per-idr` (mặc định 64).



Hình 6: Xử lý một đoạn IDR: liệt kê ứng viên, xếp hạng theo khóa, ghi bit.

8.4 Chất lượng ảnh

Lỗi native không có ngưỡng chất lượng trong bước nhúng; chất lượng được đo bên ngoài. Demo đo PSNR từng plane Y/U/V giữa `source.h264` và `stego`, in bản đồ $|\Delta Y|$. Đổi một hệ số lan truyền qua dự đoán intra sang các khối giải mã sau trong frame, bị deblocking làm mịn, và với $\text{GOP} > 1$ lan sang P-frame tham chiếu. Bộ benchmark `_new` (run 20260924T150707Z_2be60c, 30 frame đầu mỗi case) ghi nhận 13/27 case dưới ngưỡng lịch sử 40 dB của PSNR frame bị sửa (thấp nhất 32,76 dB); các kết quả này có trước giao thức v2 và cần chạy lại trước khi trích dẫn cho phiên bản hiện tại.

9 Capacity: Đếm Ứng Viên Theo Đoạn

Vì mọi ứng viên đều dùng được về mặt cú pháp, dung lượng không còn chia thành nhiều tầng lọc như bản cũ:

$$\text{capacity} = \sum_s \min(\text{max_bits_per_IDR}, \text{ứng viên}_s)$$

Một khung cần $(152 + \text{số byte message}) \times 8$ bit: 4 byte độ dài + 129 byte proof + 19 byte overhead khung (1 version + 2 độ dài + 16 tag).

Đại lượng (theo tài liệu trong repo)	Giá trị
Ứng viên của clip demo 8 frame CIF, QP 22, GOP 1	3 660
Ứng viên mỗi IDR, foreman QP 22	khoảng 450
Message 19 byte → khung	171 byte = 1 368 bit (khoảng 37% của 3 660)

Với cap mặc định 64, clip ngắn có thể không đủ dung lượng ($8 \text{ IDR} \times 64 = 512$ bit). Nếu không truyền `--max-bits-per-idr`, demo tự nâng cap lên giá trị nhỏ nhất đủ dung lượng và in lý do; nếu truyền mà thiếu thì dừng với gợi ý tăng `--frames`, giảm `--gop`, tăng cap hoặc rút ngắn message. `zkstego_inspect <video> --segments MAX_BITS` liệt kê ứng viên và dung lượng của từng đoạn (JSON `segments-1`); CLI `measure-live-capacity-stdin` in dung lượng dạng JSON từ stdin.

10 Payload Được Chuẩn Bị Như Thế Nào?

10.1 Payload

[message_length: 4 byte big-endian] [message] [proof nén: 129 byte]

Hàm `pack(message, proof_bytes)` và `unpack(blob)` nằm trong `src/zk_proof.py`. Payload tối đa 4096 byte ở các lệnh stdin (giới hạn cấu hình; định dạng khung cho phép tới 65 535), nên message tối đa $4096 - 4 - 129 = 3963$ byte.

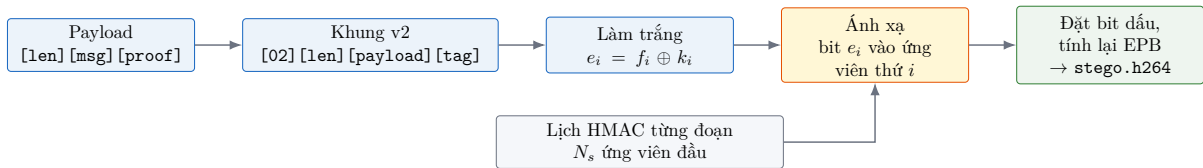
10.2 Khung xác thực v2 và làm trắng

```
[version = 0x02 : 1 byte][payload_len : 2 byte BE][payload][tag : 16 byte]
tag = HMAC-SHA256(frame_key, version || len || payload)[0:16]
keystream = HMAC(whitening_key, u64be(0)) || HMAC(whitening_key, u64be(1)) || ...
bit_nhưng[i] = bit_khung[i] XOR bit_keystream[i]
```

Chỉ số i đánh trên toàn khung, kể cả khi khung trải nhiều IDR. Không làm trắng thì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch; sau khi XOR với dòng khóa giả ngẫu nhiên, các bit nhúng phân bố như ngẫu nhiên đối với người không có khóa. Giao thức v2 không đọc được file stego tạo bởi giao thức v1.

Thành phần	Kích thước	Ghi chú
Version	1 byte	0x02
Độ dài payload	2 byte	Big-endian
Độ dài message	4 byte	Big-endian
Message	m byte	UTF-8
Proof nén	129 byte	Groth16 BN254
Tag HMAC	16 byte	frame_key
Tổng	$152 + m$ byte	$m = 19 \Rightarrow 171 \text{ byte} = 1368 \text{ bit}$

11 Payload Được Nhúng Vào Video Như Thế Nào?



Hình 7: Từ payload đến video stego: khung xác thực, làm trắng, lịch có khóa.

Lệnh native (khóa không bao giờ đi qua tham số dòng lệnh):

```
zkstego_blind_bits embed-stream-auth-stdin in.h264 out.h264 <max_bits_per_IDR>
```

stdin dòng 1 = khóa 64 ký tự hex, dòng 2 = payload hex. Mã thoát 2 cho mọi lỗi. Ngoài ra, `embed-live-auth-stdin` và `extract-live-auth-stdin` làm việc y hệt nhưng video vào/ra qua stdin/stdout (dùng cho WebSocket).

11.1 Ảnh hưởng lên một khối: ví dụ thật

Khối `luma4x4BlkIdx 2` của MB 280, frame #6 (mode Intra4x4 suy ra là 1, Horizontal; QP 19). Lật dấu trailing-one:

Hệ số trước nhúng	Hành động	Hệ số sau nhúng																																
<table><tr><td>2</td><td>-1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	2	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	trailing_ones_sign_flag 1 → 0 hệ số -1 → +1	<table><tr><td>2</td><td>+1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	2	+1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	-1	0	0																															
0	0	0	0																															
0	0	0	0																															
0	0	0	0																															
2	+1	0	0																															
0	0	0	0																															
0	0	0	0																															
0	0	0	0																															

Hình 8: Ví dụ thật: nhúng một bit bằng cách đổi dấu một hệ số trailing-one.

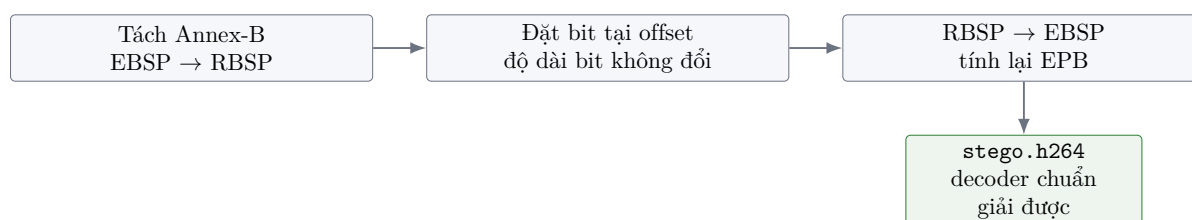
Bước (mỗi hàng của khối)	Trước	Sau
Giải lượng tử, hàng 0	[176, -112, 0, 0]	[176, 112, 0, 0]
Residual sau biến đổi ngược	[1, 2, 4, 5]	[5, 4, 2, 1]
Dự đoán Horizontal	cột trái = 83	
Tái tạo (trước deblocking)	[84, 85, 87, 88]	

Giá trị tái tạo “trước” khớp ảnh Ffmpeg `-skip_loop_filter` all từng mẫu. Sau nhúng, pixel của khối đổi [6, 4, 0, -2] mỗi hàng = Δ dự đoán [2, 2, 2, 2] + Δ residual [4, 2, -2, -4]. Δ dự đoán khác 0 vì cột mẫu bên trái (thuộc MB 279) đã đổi do một lần lật dấu giải mã trước đó: ảnh hưởng **lan truyền** qua dự đoán intra.

12 Ghi Lại Bitstream: Vì Sao Video Sau Nhúng Vẫn Là H.264 Hợp Lệ?

Bản cũ có bước “reconstruction” mã hóa lại khối CAVLC và đếm lại số vị trí “sống sót”. Lỗi hiện tại không cần bước đó:

1. Tách Annex-B thành NAL, chuyển NAL cần sửa sang RBSP.
2. Với mỗi vị trí trong lịch, **đặt** bit tại offset `rbsp_bit_offset` thành bit nhúng; độ dài RBSP không đổi. Hàm native: `apply_fixed_length_patches`.
3. Chuyển RBSP về Ebsp (chèn lại EPB) và ghép lại Annex-B.



Hình 9: Ghi lại bitstream: không mã hóa lại, chỉ đặt bit dấu rồi tính lại EPB.

Bằng chứng “vẫn hợp lệ” đến từ kiểm thử, không phải từ bộ đếm sau nhúng: phase 3 (`test_native_cli_fixture.py`) nhúng/trích trên fixture 300 frame với decode nghiêm ngặt; demo giải mã stego bằng Ffmpeg và kiểm số frame, ánh xạ 1 slice/frame, danh tính ứng viên, bit và metadata không đổi giữa trước/sau.

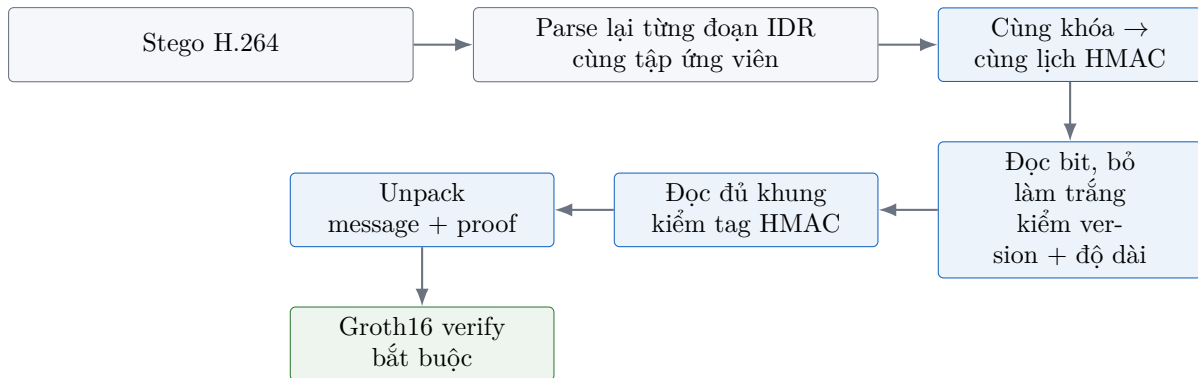
12.1 Đầu ra

Đầu ra của lệnh nhúng chỉ là **một file stego.h264**. Không có file phụ chứa vị trí. Khi chạy demo, thư mục `demo/runs/terminal_<thời gian>_<ngẫu nhiên>/` chứa thêm các artifact để quan sát (không cần cho việc trích xuất):

File	Nội dung
source.h264, stego.h264	Video trước/sau nhúng.
schedule.json	Mọi vị trí được dùng: đoạn, bit khung, ID trong đoạn, ID file, score, bit trước/đích.
trace_before.json, trace_after.json	Mọi NAL, slice, ứng viên, bit đầu, hệ số.
segments_before.json, segments_after.json	Input lịch theo đoạn (segments-1).
proof.json, proof.bin, public_signals.json	Proof và public inputs.
transcript.txt, report.json	Thuyết minh (secret bị che), kết quả kiểm tra, timing, PSNR.

13 Trích Xuất Payload Diễn Ra Như Thế Nào?

Trích xuất là mù: bên nhận chỉ cần video stego và secret, không cần video gốc, không cần file phụ.



Hình 10: Pipeline trích xuất mù: dựng lại lịch từ stego và khóa, kiểm HMAC, rồi verify proof.

1. Parse lại các IDR của video stego, liệt kê ứng viên. Tập ứng viên giống hệt phía gửi vì lật dấu không đổi TotalCoeff/TrailingOnes, tức không đổi cấu trúc.
2. Dùng cùng secret sinh lại các khóa con, tính score, sắp xếp, đọc bit tại các vị trí đã chọn (cùng `max_bits_per_IDR`).
3. Bỏ làm trắng 24 bit đầu $\rightarrow$ kiểm version = 2 và độ dài $\leq$ giới hạn $\rightarrow$ biết tổng số bit cần đọc. Sai khóa thì byte version sau khi bỏ làm trắng gần như ngẫu nhiên nên bị từ chối ngay.
4. Đọc đủ khung, bỏ làm trắng, tính lại tag và so sánh hằng thời gian.

Lệnh native:

```
zkstego_blind_bits extract-stream-auth stego.h264 - <max_payload> <max_bits_per_IDR>
```

stdin: khóa hex; stdout: payload hex (đã qua HMAC). Các thông báo lỗi native (đều thoát mã 2):

Thông báo	Khi nào
authenticated CAVLC frame tag is invalid	Đủ bit nhưng tag sai (sai khóa hoặc bit bị sửa).
authenticated CAVLC frame/stream version is invalid	Byte đầu sau khi bỏ làm trắng khác 0x02.
authenticated CAVLC frame/stream length exceeds configured maximum	Trường độ dài vượt giới hạn.
blind schedule capacity is insufficient	Lịch cần nhiều bit hơn số ứng viên của đoạn.
authenticated CAVLC stream is incomplete (và các biến thể “IDR stream ended before ...”)	Hết video trước khi đọc đủ khung.

HMAC hợp lệ chỉ chứng minh payload đến từ người có khóa và không bị sửa; **không** thay cho việc kiểm proof Groth16. CLI native tự nó chỉ kiểm HMAC; quy tắc “proof bắt buộc” do demo và job /api/v1/jobs/verify thực thi ngay sau trích xuất.

14 Cơ Chế ZKP: Proof Được Sinh Từ Đâu Và Sinh Như Thế Nào?

ZKP không chọn vị trí nhúng trong video. ZKP tạo ra một proof mật mã để chứng minh rằng người tạo biết một secret khớp với commitment của message, nhưng không đưa secret vào public signals. Proof được nén thành 129 byte, đóng gói vào payload, và đối với lỗi media nó chỉ là dữ liệu cần giấu và đọc lại chính xác.

14.1 Mệnh đề mà proof chứng minh

Circuit PayloadVerify trong `circuits/payload_verify.circom` chứng minh:

$$\text{commitment} = \text{SHA256}(\text{payload_hash} \parallel \text{secret}), \quad 0 < \text{payload_length} < 1\,000\,000$$

với $\text{payload_hash} = \text{SHA256}(\text{message})$ được tính **ngoài** mạch. Thêm vào đó, mọi tín hiệu của `payload_hash`, `commitment` và `secret` bị ràng buộc là bit: $x(x-1) = 0$. Ràng buộc này được thêm ngày 03/10/2026 vì thành phần `Sha256` của `circomlib` không tự ép đầu vào là 0/1; proof tạo bằng khóa của mạch cũ không verify được với khóa hiện tại. Mạch có **63 321 constraint**.

Mạch không chứa hash video, chỉ số frame hay danh tính camera.

14.2 Public input và private input

Loại input	Tên	Ý nghĩa
Public	<code>payload_hash[256]</code>	256 bit của $\text{SHA256}(\text{message})$.
Public	<code>commitment[256]</code>	256 bit commitment.
Public	<code>payload_length</code>	Độ dài message theo byte.
Private	<code>secret[256]</code>	256 bit secret, tức 32 byte.

Tổng cộng $256 + 256 + 1 = 513$ public input.

14.3 Dữ liệu trước khi tạo proof

Hàm `_build_circuit_input(payload_bytes, secret_key)` của `ZKSnarkBridge` trong `src/zk_proof.py`:

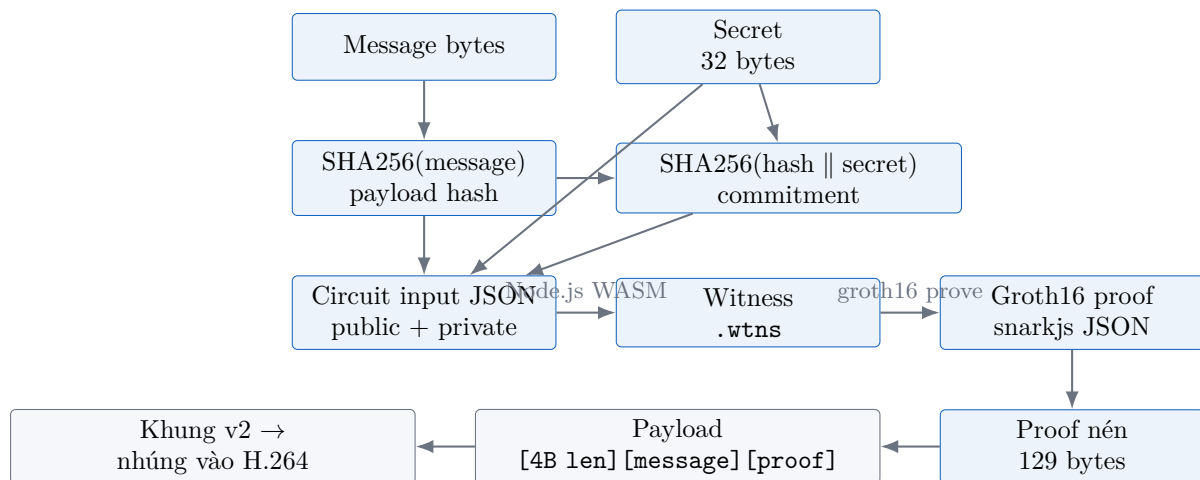
Bước	Giá trị tạo ra
1	Kiểm <code>secret_key</code> đúng 32 byte.
2	<code>payload_hash = SHA256(message)</code>
3	<code>commitment = SHA256(payload_hash secret_key)</code>
4	Chuyển <code>payload_hash</code> , <code>commitment</code> , <code>secret_key</code> thành mảng 256 bit.
5	Ghi thêm <code>payload_length = len(message)</code> .

14.4 Workflow sinh proof

Hệ thống dùng `snarkjs` và `Groth16`. Các artifact nằm trong `circuits/build/` (bị git-ignore, tạo bằng `py -3.12 demo/terminal_demo.py --setup`):

Artifact	Vai trò
<code>payload_verify_js/payload_verify.wasm</code>	Chạy circuit để tạo witness từ input.
<code>payload_verify_js/generate_witness.js</code>	Script Node.js sinh witness.
<code>proving_key.zkey</code>	Proving key dùng để tạo Groth16 proof.
<code>verification_key.json</code>	Verification key dùng để verify proof.
<code>DEMO_SETUP_ONLY.txt</code>	Đánh dấu khóa là thiết lập giáo dục cục bộ.

Khóa proving/verification hiện là **thiết lập cục bộ dành cho demo**, không phải trusted ceremony cho triển khai.



Hình 11: Workflow tạo proof: message và secret tạo witness, witness tạo Groth16 proof, proof được nén rồi đóng gói vào payload.

14.5 Witness là gì?

Witness là tập giá trị làm cho circuit thỏa mãn: secret và các giá trị trung gian khi circuit tính `SHA256(payload_hash || secret)`. Nếu secret không đúng, không thể tạo witness hợp lệ cho commitment đã công khai.

Witness được tạo bằng `node generate_witness.js payload_verify.wasm input.json witness.wtns` trong một thư mục tạm dưới `circuits/build/`. Input JSON chứa secret ở dạng bit, nên các file tạm (input, witness, proof, public) được xóa trong khối `finally`; thư mục tạm cũ hơn 1 giờ do lần chạy bị dừng đột ngột được dọn khi khởi tạo bridge.

14.6 Proof JSON và proof bytes khác nhau thế nào?

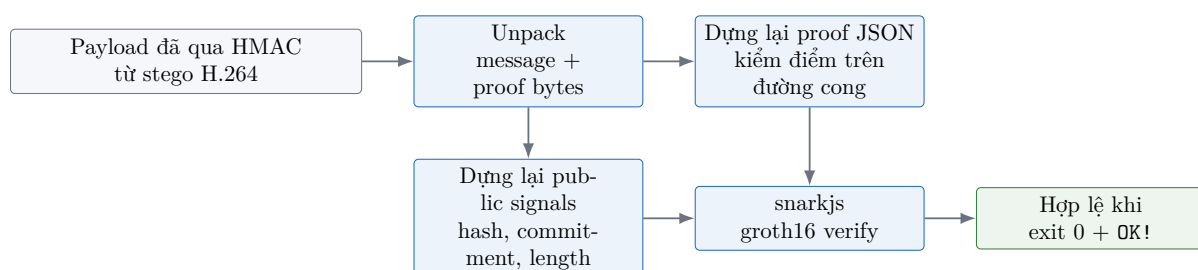
snarkjs tạo proof ở dạng JSON gồm `pi_a` ($A \in G_1$), `pi_b` ($B \in G_2$), `pi_c` ($C \in G_1$), `protocol = groth16`, `curve = bn128` (tức BN254). Hệ thống nén proof thành 129 byte bằng `proof_to_bytes`:

Thành phần	Kích thước
<code>pi_a.x</code>	32 byte
<code>pi_b.x[0]</code>	32 byte
<code>pi_b.x[1]</code>	32 byte
<code>pi_c.x</code>	32 byte
Byte cờ: 3 bit chẵn/lẻ của tọa độ y (A, B, C)	1 byte
Tổng	129 byte

Khi giải nén, `bytes_to_proof` kiểm tra: đúng 129 byte, không có bit cờ lạ, tọa độ nhỏ hơn p , và điểm nằm trên đường cong. Dữ liệu hỏng $\rightarrow$ `ValueError` $\rightarrow$ verifier trả `valid = False`.

14.7 Verify sau khi extract diễn ra thế nào?

1. Trích xuất mù và kiểm HMAC (mục trước) để có payload.
2. `unpack`: đọc 4 byte độ dài, tách message và 129 byte proof.
3. Dựng lại proof JSON bằng `bytes_to_proof()` (có kiểm điểm).
4. Tính lại `payload_hash` và `commitment` từ message và secret.
5. Tạo 513 public signal.
6. Gọi `snarkjs groth16 verify` với `verification_key.json`; chỉ chấp nhận khi mã thoát 0 và có dòng OK!.



Hình 12: Workflow verify: sau khi trích đúng bytes, hệ thống dựng lại proof và public signals rồi gọi Groth16 verifier.

14.8 Proof chứng minh gì, không chứng minh gì

- Chứng minh: người tạo proof biết **secret** sao cho commitment khớp message.
- Verifier phải biết **secret** để tự tính lại commitment, nên trong cấu hình hiện tại ai có khóa cũng tạo được proof.
- **Không** ràng buộc proof với video, frame, thời điểm hay camera: một payload có thể bị chép sang video khác dùng cùng khóa. Hướng khắc phục nằm trong `future_plan.md`.

Thành phần	Trách nhiệm
ZKP layer	Chứng minh message có commitment hợp lệ với secret mà không đưa secret vào public signals.
Kênh xác thực	Chọn vị trí theo khóa, làm trắng bit, phát hiện sai khóa hoặc bit bị sửa (HMAC).
Lỗi media	Đặt và đọc bit dấu mà bitstream vẫn hợp lệ.
Verifier	Kiểm tra bytes đọc lại tạo thành proof Groth16 hợp lệ trước khi dùng message.

15 Luồng Thực Tế: CLI, Dịch Vụ HTTP/WebSocket Và Demo

15.1 Luồng đầu-cuối

Gửi:

1. Encode video về Baseline/CAVLC.
2. Sinh proof bằng `generate_proof_for_payload(message, secret)` của ZKSnarkBridge, rồi nén bằng `proof_to_bytes` (129 B).
3. `pack(message, proof_bytes) → payload`.
4. `zkstego_blind_bits embed-stream-auth-stdin`.

Nhận:

1. `zkstego_blind_bits extract-stream-auth` → payload đã qua HMAC.
2. `unpack` → (message, proof 129 B) → `bytes_to_proof`.
3. `verify_proof_for_payload(proof, message, secret)`; chỉ khi đúng mới dùng message.

15.2 Các lệnh native

Lệnh	Mục đích
<code>zkstego_blind_bits embed-stream-auth-stdin, extract-stream-auth</code>	Khung xác thực, lịch theo đoạn IDR, đọc/ghi file.
<code>zkstego_blind_bits measure-live-capacity-stdin</code>	Đọc Annex-B từ stdin, in JSON dung lượng ứng viên.
<code>zkstego_blind_bits embed-live-auth-stdin, extract-live-auth-stdin</code>	Như chế độ đoạn nhưng video vào/ra qua stdin/stdout.
<code>zkstego_inspect <video></code>	Công cụ soi duy nhất: JSON NAL/slice/ứng viên.
<code>zkstego_inspect <video> --summary</code>	Tóm tắt dễ đọc.
<code>zkstego_inspect <video> --macroblock NAL MB</code>	Header mọi MB của slice và từng phần tử CAVLC của một MB.
<code>zkstego_inspect <video> --segments MAX_BITS</code>	Ứng viên theo đoạn (<code>segments-1</code>).

15.3 Dịch vụ HTTP/WebSocket

Một ứng dụng: `src.api.native_handlers:create_native_app`, dựng trên lõi job dùng chung `src.api.app.create_app`.

Endpoint	Mô tả
GET /health	Công khai.
POST /api/v1/jobs/embed	Upload .h264 + message_b64 + secret_key_b64 → job id.
POST /api/v1/jobs/extract	Upload stego + khóa → job id.
POST /api/v1/jobs/verify	Trích mù → unpack → Groth16 verify bắt buộc . succeeded kèm message chỉ khi proof đúng; ngược lại rejected (payload_not_authenticated, malformed_proof_payload, proof_invalid); lỗi hạ tầng → failed.
GET /api/v1/jobs/{id}	Trạng thái job.
GET /api/v1/jobs/{id}/artifact	Tải kết quả một lần, hết hạn sau 10 phút.
WS /api/v1/stream	Khung JSON start, rồi các chunk Annex-B nhị phân.

Bảo vệ: Bearer token ZK_STEGO_API_TOKEN bắt buộc, tối thiểu 32 ký tự, được kiểm **trước khi đọc body**; 10 lần xác thực sai từ một IP trong 60 s → 429 kèm **Retry-After** (cả HTTP lẫn WebSocket); tối đa 4 upload đồng thời; hàng đợi job có giới hạn; khóa đi vào tiến trình native qua stdin, không qua shell.

15.4 Quan sát bằng demo

Demo duy nhất là demo/terminal_demo.py:

```
py -3.12 demo/terminal_demo.py --setup
py -3.12 demo/terminal_demo.py --auto --message "Xin chào"
py -3.12 demo/terminal_demo.py --auto --brief --no-service
py -3.12 demo/terminal_demo.py --auto --frames 48 --gop 12
```

Các bước liên quan trực tiếp đến tài liệu này: bên trong FFmpeg/libx264 (log verbose, thống kê x264); bảng NAL, EBSP→RBSP và header do `trace_headers` giải; dung lượng theo đoạn và lịch (khóa con HKDF, khung v2, bảng IDR nào mang khoảng bit khung nào, score HMAC, bit khung $\oplus$ keystream); nhúng native; giải phần một khối (đối chiếu từng mẫu với ảnh FFmpeg `-skip_loop_filter all`); bit nhúng nằm ở đâu; verifier với Groth16 bắt buộc; các ca từ chối (đối message, proof sai/thiếu, sai khóa, lật một bit mang dữ liệu trong file); job HTTP và luồng WebSocket. `--brief` bỏ các bước giải thích sâu, `--no-service` bỏ hai bước dịch vụ. Mọi kiểm tra in `[PASS]/[FAIL]`; một `[FAIL]` dừng demo với mã thoát 1.

16 Vì Sao Chỉ IDR Mang Dữ Liệu?

Parser native chỉ giải slice IDR (I-slice). Với `-g 1` mọi frame là IDR, nên mọi frame mang ứng viên; đây là cấu hình mặc định của demo và fixture kiểm thử. Lý do:

- residual của I-slice chỉ phụ thuộc dự đoán intra trong cùng frame;
- mỗi IDR cùng SPS/PPS tạo thành một đoạn độc lập, nên lịch có thể tính theo từng đoạn, phù hợp cho cả file lẫn luồng live;
- CAVLC Baseline có cấu trúc trailing-one phù hợp với kênh dấu.

Với `GOP>1` (ví dụ `--gop 12 --frames 48`), hệ thống vẫn chạy nhưng P-frame không có ứng viên, nên dung lượng chỉ đến từ IDR; thay đổi trong IDR còn lan sang các P-frame tham chiếu nó.

17 Tóm Tắt Vai Trò Của Các Thành Phần

Đường dẫn	Vai trò
native/include/zkstego/cavlc_stream.hpp, native/src/cavlc_stream.cpp	Parser H.264/CAVLC, ứng viên, HKDF, lịch HMAC, khung, encoder/decoder theo đoạn.
native/tools/blind_bits.cpp	CLI nhúng/trích zkstego_blind_bits.
native/tools/inspect.cpp	Công cụ soi duy nhất zkstego_inspect.
native/tests/cavlc_stream_tests.cpp	CTest native.
src/native_blind_contract.py	Hợp đồng lịch/khung bằng Python (tham chiếu), segment_schedule.
src/zk_proof.py, circuits/payload_verify.circom	Proof Groth16, nén, pack/unpack, xác minh.
src/api/app.py, src/api/native_handlers.py	Lỗi job dùng chung + dịch vụ HTTP/WebSocket.
src/realtime_cavlc.py	Bộ lập lịch realtime theo đoạn.
src/h264_tables.py	Bảng VLC CAVLC chuẩn (cho phần giải thích của demo).
demo/terminal_demo.py và deep_trace, h264_explain, segment_plan, negative_cases, service_steps, native_io	Demo duy nhất.
src/runtest/	Bộ test run_all.py và prepare_fixtures.py.

Lịch sử: trước 03/10/2026 repository còn một pipeline Python song song (src/embedder.py, src/verifier*.py, src/core/, src/bitstream/, chaos map, file phụ positions.json, benchmark SEC1–SEC10). Nó đã bị gỡ (vẫn xem được trong lịch sử git) và không còn là bằng chứng hiện hành.

18 Kiểm Thử, Hiệu Năng Và Giới Hạn

- **Kiểm thử:** ngày 03/10/2026, py -3.12 src/runtest/run_all.py qua **126/126 trên 12 phase** (Windows, fixture từ prepare_fixtures.py), CTest native 1/1. E2E camera vật lý (--hardware) không nằm trong lần chạy này. Mã thoát run_all: 0 = tất cả qua, 1 = có lỗi, 2 = có ca bị bỏ qua.
- **Realtime chưa được chấp nhận:** E2E camera thật đo được 7,608 FPS nguồn so với cổng 30 FPS; số đo này có trước đợt tăng tốc native ngày 02/10/2026 và chưa được đo lại. Chưa có kết quả trên thiết bị edge.
- **Đầu vào:** chỉ Baseline, CAVLC, progressive, 4:2:0, một slice/frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id; chỉ IDR mang dữ liệu.
- **Ràng buộc proof:** khung và proof không chứa định danh video hay bộ đếm, nên có thể phát lại sang video khác cùng khóa; mạch dùng secret gốc và verifier phải giữ secret. Kế hoạch: future_plan.md.

19 Điểm Cần Nhớ

Điểm tinh túy của hệ thống không phải là “đổi hệ số”. Điểm tinh túy là:

Chỉ chạm vào một bit mà cú pháp CAVLC cho phép đổi mà không ảnh hưởng gì khác, để khóa bí mật quyết định bit nào mang dữ liệu, và bắt mọi thứ đọc ra phải qua HMAC rồi qua Groth16 trước khi được tin.

Các thuật ngữ quan trọng nhất:

- **Ứng viên:** bit dấu của trailing-one đầu tiên trong một khối residual của slice IDR; một ứng viên mỗi khối.
- **Bất biến độ dài:** lật dấu không đổi TotalCoeff/TrailingOnes, nên không đổi nC, độ dài bit RBSP hay cú pháp phía sau.
- **Lịch có khóa theo đoạn:** HMAC(schedule_key, định danh) xếp hạng ứng viên trong mỗi đoạn SPS+PPS+IDR, tối đa max_bits_per_IDR bit/đoạn.
- **Khung v2 và làm trắng:** [0x02] [len] [payload] [tag], XOR với keystream HMAC.
- **Trích xuất mù:** không cần video gốc hay file phụ.
- **Groth16 bắt buộc:** HMAC đúng chưa đủ; proof phải verify (exit 0 + OK!).

20 Kết Luận

Hệ thống xử lý video theo hướng rất cụ thể:

H.264 Baseline $\rightarrow$ đoạn IDR $\rightarrow$ ứng viên dấu T1 $\rightarrow$ lịch HMAC $\rightarrow$ bit khung $\oplus$
keystream $\rightarrow$ stego H.264

Nó không giấu tin bằng cách sửa pixel. Nó giấu tin bằng cách đặt bit dấu của một số hệ số trailing-one do khóa chọn, trong một lõi C++ duy nhất. Tính hợp lệ của bitstream đến từ cấu tạo của CAVLC; tính bí mật và toàn vẹn của kênh đến từ HKDF, HMAC và làm trắng; còn quan hệ giữa message và secret được chứng minh bằng Groth16. Những gì hệ thống *chưa* làm — gán proof với video, đạt realtime trên camera thật — được ghi rõ ở trên và trong future_plan.md.